

Apostila 2 — pandas para Economia

1. O que é pandas?

pandas é a biblioteca para manipular dados tabulares (planilhas). Os dois objetos principais:

- **Series** — uma coluna (como uma lista com índice)
- **DataFrame** — tabela inteira (várias colunas)

```
import pandas as pd
```

2. Criando DataFrames

A partir de um dicionário

```
import pandas as pd
```

```
dados = {  
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia"],  
    "pib": [2.2, -1.8, 3.1, 2.8],  
    "inflacao": [4.5, 98.0, 3.8, 7.2],  
    "populacao": [214, 46, 19, 52] # milhões  
}
```

```
df = pd.DataFrame(dados)  
print(df)
```

Saída:

	pais	pib	inflacao	populacao
0	Brasil	2.2	4.5	214
1	Argentina	-1.8	98.0	46
2	Chile	3.1	3.8	19
3	Colombia	2.8	7.2	52

A partir de CSV

```
df = pd.read_csv("dados.csv")
```

A partir de Excel

```
df = pd.read_excel("dados.xlsx", sheet_name="Planilha1")
```

A partir da área de transferência

Copie uma tabela do Excel e rode:

```
df = pd.read_clipboard()
```

3. Explorando o DataFrame

```
df.head()           # primeiras 5 linhas  
df.head(10)         # primeiras 10  
df.tail(3)          # últimas 3  
df.sample(5)        # 5 linhas aleatórias
```

```
df.shape            # (número de linhas, número de colunas)  
df.info()           # tipos de dados + valores nulos  
df.describe()       # estatísticas descritivas  
df.dtypes           # tipo de cada coluna  
df.columns          # nomes das colunas  
df.index            # índice
```

```
df["pais"].unique() # valores únicos
df["pais"].value_counts() # contagem de cada valor
```

4. Selecionando dados

4.1. Colunas

```
# Uma coluna (retorna Series)
df["pais"]
df.pais # atalho (funciona só se nome não tiver espaço)

# Várias colunas (retorna DataFrame)
df[["pais", "pib"]]
```

4.2. Linhas por índice (iloc)

```
df.iloc[0] # primeira linha
df.iloc[-1] # última linha
df.iloc[0:3] # linhas 0, 1, 2
df.iloc[[0, 2]] # linhas 0 e 2

df.iloc[0, 1] # linha 0, coluna 1
df.iloc[0:3, 0:2] # linhas 0-2, colunas 0-1
```

4.3. Linhas por rótulo (loc)

```
df.index = ["a", "b", "c", "d"]
df.loc["a"] # linha com índice "a"
df.loc["a":"c"] # linhas "a" até "c"
df.loc[:, "pib"] # todas as linhas, coluna "pib"
```

4.4. Seleção mista

```
df.at[0, "pib"] # valor único (mais rápido)
df.iat[0, 1] # valor único por posição
```

5. Filtrando linhas

Essa é a operação MAIS importante. A sintaxe é:

```
df[df["coluna"] condicao]
```

Exemplos:

```
# Filtrar por valor
df[df["pais"] == "Brasil"]
df[df["pib"] > 2.0]
df[df["inflacao"] < 10]

# Múltiplas condições
df[(df["pib"] > 0) & (df["inflacao"] < 10)]

# | significa "ou"
df[(df["pib"] > 3) | (df["pais"] == "Argentina")]

# Negação
df[~(df["pais"] == "Brasil")]

# isin (vários valores)
```

```
df[df["pais"].isin(["Brasil", "Chile"])]

# between (entre dois valores)
df[df["pib"].between(0, 3)]

# str.contains (texto)
df[df["pais"].str.contains("Bra")]
```

Como funciona por baixo:

```
# O pandas cria uma máscara booleana:
mascara = df["pib"] > 2.0
print(mascara)
# 0      True
# 1     False
# 2      True
# 3      True
```

```
# Depois aplica:
df[mascara]
```

6. Criando e modificando colunas

```
# Criar nova coluna
df["pib_per_capita"] = df["pib"] / df["populacao"] * 1000
df["log_pib"] = np.log(df["pib"] + 10)

# Atualizar coluna existente
df["inflacao"] = df["inflacao"] / 100 # vira decimal

# apply – aplicar função em cada linha
df["classificacao"] = df["pib"].apply(
    lambda x: "Crescendo" if x > 0 else "Recessão"
)

# apply com função definida
def classificar_pib(x):
    if x > 3: return "Alto"
    elif x > 0: return "Médio"
    else: return "Negativo"

df["categoria"] = df["pib"].apply(classificar_pib)

# Renomear colunas
df.rename(columns={"pib": "crescimento_pib"}, inplace=True)

# Remover coluna
df.drop("coluna_que_nao_quero", axis=1, inplace=True)

# Reordenar colunas
df = df[["pais", "pib", "inflacao", "populacao"]]
```

7. Valores nulos

Dados reais SEMPRE têm valores faltando.

```

# Verificar nulos
df.isnull().sum()          # quantos nulos por coluna
df.isnull().sum().sum()    # total de nulos

# Remover nulos
df.dropna()                # remove linhas com qualquer nulo
df.dropna(subset=["pib"])   # remove só se "pib" for nulo

# Preencher nulos
df.fillna(0)               # preenche com zero
df.fillna(df.mean())       # preenche com a média
df["pib"].fillna(method="ffill") # preenche com último valor (série temporal)

```

8. Agrupamento (groupby)

Equivalente à tabela dinâmica do Excel.

```

# Criando dados para exemplo
dados = {
    "ano": [2020, 2020, 2020, 2021, 2021, 2021],
    "pais": ["Brasil", "Argentina", "Chile", "Brasil", "Argentina", "Chile"],
    "pib": [-3.3, -9.9, -6.1, 4.8, 10.4, 11.7]
}
df = pd.DataFrame(dados)

# Agrupar e calcular
df.groupby("pais")["pib"].mean()
df.groupby("pais")["pib"].agg(["mean", "min", "max", "std"])

# Agrupar por múltiplas colunas
df.groupby(["ano", "pais"])["pib"].sum()

# Várias agregações
df.groupby("pais").agg({
    "pib": ["mean", "std"],
    "ano": "count"
})

```

9. Juntar tabelas (merge)

Como PROCX do Excel, mas muito mais poderoso.

```

pib_df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile"],
    "pib": [2.2, -1.8, 3.1]
})

inf_df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile"],
    "inflacao": [4.5, 98.0, 3.8]
})

# Inner join (só o que existe nas duas)
merged = pd.merge(pib_df, inf_df, on="pais")

# Left join (tudo da esquerda)
merged = pd.merge(pib_df, inf_df, on="pais", how="left")

```

```
# Outer join (tudo dos dois)
merged = pd.merge(pib_df, inf_df, on="pais", how="outer")

# Quando os nomes das colunas são diferentes:
# pd.merge(df1, df2, left_on="pais", right_on="country")
```

10. Trabalhando com datas

Essencial para séries temporais.

```
# Criando datas
df["data"] = pd.to_datetime(df["data"])
df["ano"] = df["data"].dt.year
df["mes"] = df["data"].dt.month
df["trimestre"] = df["data"].dt.quarter

# Filtrar por período
df[df["data"] >= "2020-01-01"]
df[(df["data"] >= "2020-01-01") & (df["data"] <= "2022-12-31")]

# Usar data como índice
df.set_index("data", inplace=True)
df.loc["2020"] # dados de 2020 inteiro
df.loc["2020-01":"2020-06"] # primeiro semestre de 2020
df.loc["2020-01-15":"2020-03-15"]

# Reamostragem (mudar frequência)
df.resample("Y").mean() # anual (Y = year)
df.resample("Q").mean() # trimestral
df.resample("M").mean() # mensal
df.resample("D").mean() # diário
```

Operações com lag/defasagem

```
# Lag (valor do período anterior)
df["pib_lag1"] = df["pib"].shift(1)
df["pib_lag2"] = df["pib"].shift(2)

# Diferença
df["pib_diff"] = df["pib"].diff() # diferença absoluta
df["pib_pct"] = df["pib"].pct_change() * 100 # variação percentual

# Média móvel
df["pib_mm3"] = df["pib"].rolling(window=3).mean()
df["pib_mm12"] = df["pib"].rolling(window=12).mean()
```

11. Estatísticas descritivas

```
df["pib"].mean() # média
df["pib"].median() # mediana
df["pib"].std() # desvio padrão
df["pib"].var() # variância
df["pib"].min() # mínimo
df["pib"].max() # máximo
df["pib"].quantile(0.25) # 1º quartil
df["pib"].quantile([0.25, 0.5, 0.75]) # vários quartis
```

```

df["pib"].skew()          # assimetria
df["pib"].kurtosis()      # curtose
df["pib"].corr(df["inflacao"]) # correlação

# Matriz de correlação
df[["pib", "inflacao", "populacao"]].corr()

# Tabela de correlação formatada
corr = df.corr(numeric_only=True)
print(corr.round(2))

```

12. Salvando dados

```

df.to_csv("dados_processados.csv", index=False, decimal=".", sep=";")
df.to_excel("dados_processados.xlsx", sheet_name="Dados", index=False)
df.to_csv("dados_processados.csv", index=False)

```

13. Exemplo completo: script real

Aqui está um script que faz o que um economista faz todo dia:

```

import pandas as pd
from bcb import sgs

# 1. Baixar dados
ipca = sgs.get({"IPCA": 433}, start="2000-01-01")
selic = sgs.get({"SELIC": 11}, start="2000-01-01")

# 2. Juntar
df = pd.merge(ipca, selic, on="Date")
df.index = pd.to_datetime(df.index)

# 3. Criar colunas
df["juro_real"] = df["SELIC"] - df["IPCA"]
df["ipca_acum"] = (1 + df["IPCA"] / 100).cumprod() - 1
df["ipca_acum"] *= 100

# 4. Filtrar
recentes = df.loc["2023:"]

# 5. Estatísticas
print("Correlação IPCA x SELIC (2023+):")
print(recentes.corr().round(3))

# 6. Salvar
recentes.to_csv("analise_bcb.csv", decimal=".",)
print("Arquivo salvo: analise_bcb.csv")

```

14. Exercícios

Exercício 1

Crie um DataFrame com PIB, inflação e desemprego de 5 países de sua escolha. Calcule a correlação entre PIB e inflação.

Exercício 2

Baixe o IPCA (código 433) e a SELIC (código 11) de 2010 até hoje. Crie uma coluna com a taxa de juros real (SELIC - IPCA).

Exercício 3

Use a base do Banco Mundial (WDI) para baixar PIB per capita de 10 países. Filtre apenas países com PIB per capita acima da média.

Exercício 4

Baixe o IPCA acumulado em 12 meses (código 13522). Calcule média móvel de 3 meses e salve em CSV.

Solução Exercício 2

```
from bcb import sgs

ipca = sgs.get({"IPCA": 433}, start="2010-01-01")
selic = sgs.get({"SELIC": 11}, start="2010-01-01")

df = pd.merge(ipca, selic, on="Date")
df["juro_real"] = df["SELIC"] - df["IPCA"]
df.columns = ["ipca", "selic", "juro_real"]

print(df.tail())
df.to_csv("juro_real.csv", decimal=",")
```

15. Resumo pandas

Operação	Código
Abrir CSV	<code>pd.read_csv("arquivo.csv")</code>
Ver primeiras linhas	<code>df.head()</code>
Filtrar linhas	<code>df[df["coluna"] > 0]</code>
Selecionar colunas	<code>df[["col1", "col2"]]</code>
Nova coluna	<code>df["nova"] = df["x"] * 2</code>
Agrupar	<code>df.groupby("x")["y"].mean()</code>
Juntar tabelas	<code>pd.merge(df1, df2, on="x")</code>
Salvar	<code>df.to_csv("saida.csv")</code>
Data como índice	<code>df.set_index("data")</code>
Lag	<code>df["x"].shift(1)</code>
Média móvel	<code>df["x"].rolling(3).mean()</code>
Correlação	<code>df.corr()</code>

Próxima apostila: Visualização de dados

Quando dominar pandas, me chame para a **Apostila 3 — matplotlib e seaborn**. Você vai aprender a transformar esses dados em gráficos prontos para publicação.